

Web Development Concepts for Everyone

By Scott Bromander

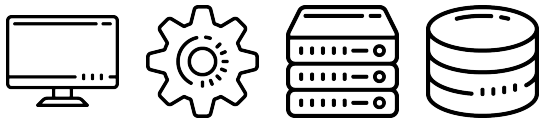


A Quick Note



01

The Full Stack



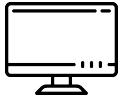
Full Stack

Layers of a Web Application (Web App)

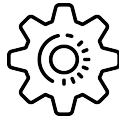
The 'Full Stack' is all the technology needed to make an app work. It includes the parts you see and interact with, along with the behind-the-scenes technology that manages the data and keeps everything running smoothly.

THE FULL STACK

Layers of a Web Application (Web App)



Front End



Client



Server



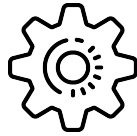
Database



Front End

The Visual Interface

The Front End is everything you see on a website, like the layout and design. It's created HTML and CSS, which help organize and style the website's content.



Client

Processing and Interaction Logic

The Client is the logic that makes a webpage “smart”. It handles User Interactions and the flow of data between the User and the Server.

The Client is what turns a Web Page into a Web Application.



Server

Backbone of Application Communication

A pivotal component that handles all data transactions between the user-facing web application on your device and the database that stores all the application's data.



Database

Application Storage Solution

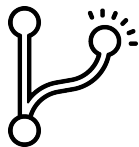
Stores data for a web application long-term. Unlike browsers, which can only hold information temporarily, databases maintain persistent data.

This means the data remains intact even if the web application (web page) is closed.



02

Version Control



Version Control

Tracking Progress, Empowering Collaboration

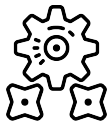
Version Control is a technology that tracks and manages changes to software as it is developed.

It goes beyond simply saving files by allowing developers to experiment safely, revert to previous versions, and collaborate effectively with others.

This ensures that progress is organized and the development process remains smooth and efficient.

VERSION CONTROL HELPS DEVELOPERS WORK TOGETHER

With Version Control, developers can:



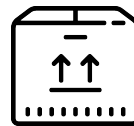
Collaborate

Developers can work on the same code base at the same time.



Isolate

Isolate "branches" of their code to try out and solidify ideas.



Deploy

Use the technology to deploy to multiple locations

Version control is critical to modern multi-developer development



Git

Powering Collaboration and Code Management

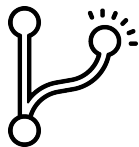
Git is a powerful version control technology that enables features like version branching, allowing developers to experiment and collaborate effectively. It works seamlessly with popular platforms like GitHub, Bitbucket, and GitLab to streamline workflows and manage code.



Commit

Capturing Progress, One Snapshot at a Time

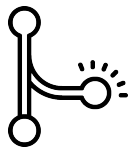
A commit is a "snapshot" of a codebase that a developer creates at a chosen point in time. It captures the state of the code as they work through an idea, typically when part or all of the idea is established and functional.



Branch

Flexible Paths for Feature Development

A branch is a copy of the codebase at a specific commit, including all previous commits. It creates a new "timeline" that allows developers to work on the code in isolation, enabling experimentation or development of new features without affecting the main codebase.



Merge

Bringing Work Together Without Losing History

Merge combines changes from one branch into another by creating a new commit that connects both histories. It keeps all original commits intact and may require resolving conflicts if both branches changed the same parts of a file.



GitHub

Where Code Meets Community

GitHub is a widely-used cloud platform built around Git, popular among individual developers and companies alike. It allows developers to store, share, and collaborate on code, with additional tools to streamline development workflows. GitHub was acquired by Microsoft in 2018.



03

Programming Languages



Runtime Environments

Where Code Comes to Life

A Runtime Environment is the environment where an application, created from written code, is executed. For JavaScript, examples include Node.js, used for server-side development, and web browsers like Chrome, which execute JavaScript on the client side. Each serves a distinct purpose in application execution.

EXAMPLES OF WHERE SOME LANGUAGES “RUN”

Spring (Server)
Android (Mobile)

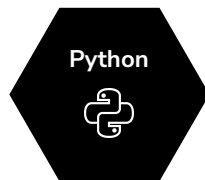


Swift

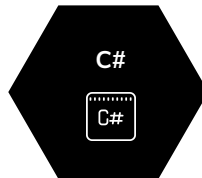


iOS (Mobile)

Flask (Server)
FastAPI (Server)
Django (Server)

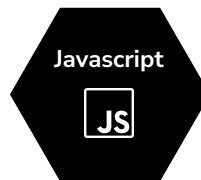


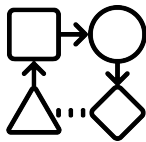
C#



.NET (Server)
Unity (Game)

Chrome (Front End)
Express (Server)





Types in Programming

Defining Information in Programming

One of the key differences between programming languages is how they handle "types." Types are classifications that describe the kind of data a piece of information represents.

Common examples of basic types in programming include **Numbers**, **Strings**, and **Booleans**.

PROGRAMMING TYPES: WEAK VS. STRONG



Weak

Can change Type

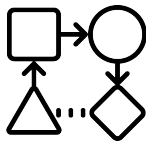
A unit of information, often called a "variable," can change from one type to another within a computer program.



Strong

Cannot change Type

A variable is not allowed to change its type within a program. Once declared, the variable is "locked" to its type, and attempting to change it will result in an error.



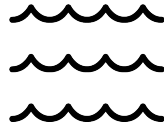
Types in Programming

Defining Information in Programming

One of the key differences between programming languages is how they handle "types." Types are classifications that describe the kind of data a piece of information represents.

Common examples of basic types in programming include **Numbers**, **Strings**, and **Booleans**.

PROGRAMMING TYPES: DYNAMIC VS. STATIC



Dynamic

Runtime Errors

Errors are revealed while an application is running, allowing developers to identify issues as they occur. This can be an advantage, as it gives developers the opportunity to iterate and "discover" their application through real-time feedback.

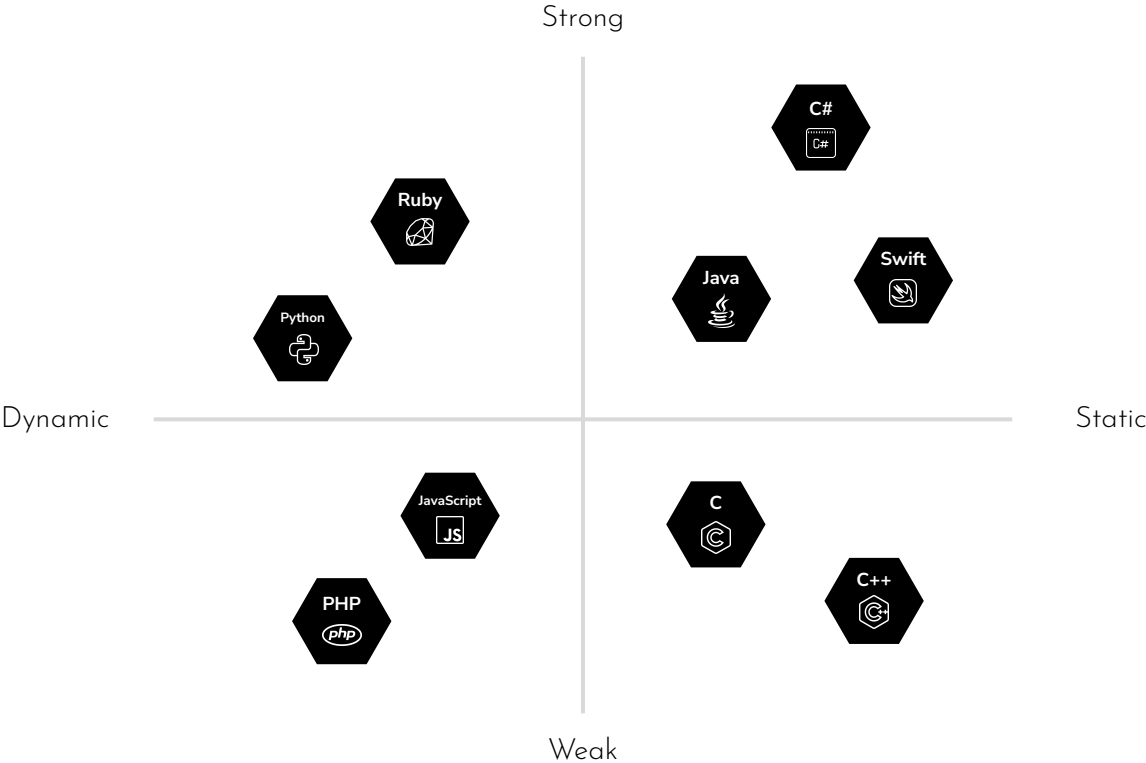


Static

Compile Time Errors

Compile-time error checking validates types while the application is being compiled, providing developers with an early warning that their logic may contain flaws.

Strong vs. Weak | Dynamic vs. Static





Supersets

Expanding on the Foundations of a Language

A superset language is a programming language that includes all the features of another language and extends it with additional capabilities and enhancements.



TypeScript

An example of a Common Superset

TypeScript is a strict syntactical superset of JavaScript, developed and maintained by Microsoft. It enhances JavaScript by adding optional static typing and configurable features, allowing seamless integration with existing JavaScript code.

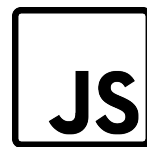


TypeScript

More features for developers

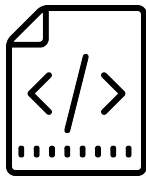


transpiles to



JavaScript

Can be read natively by the browser

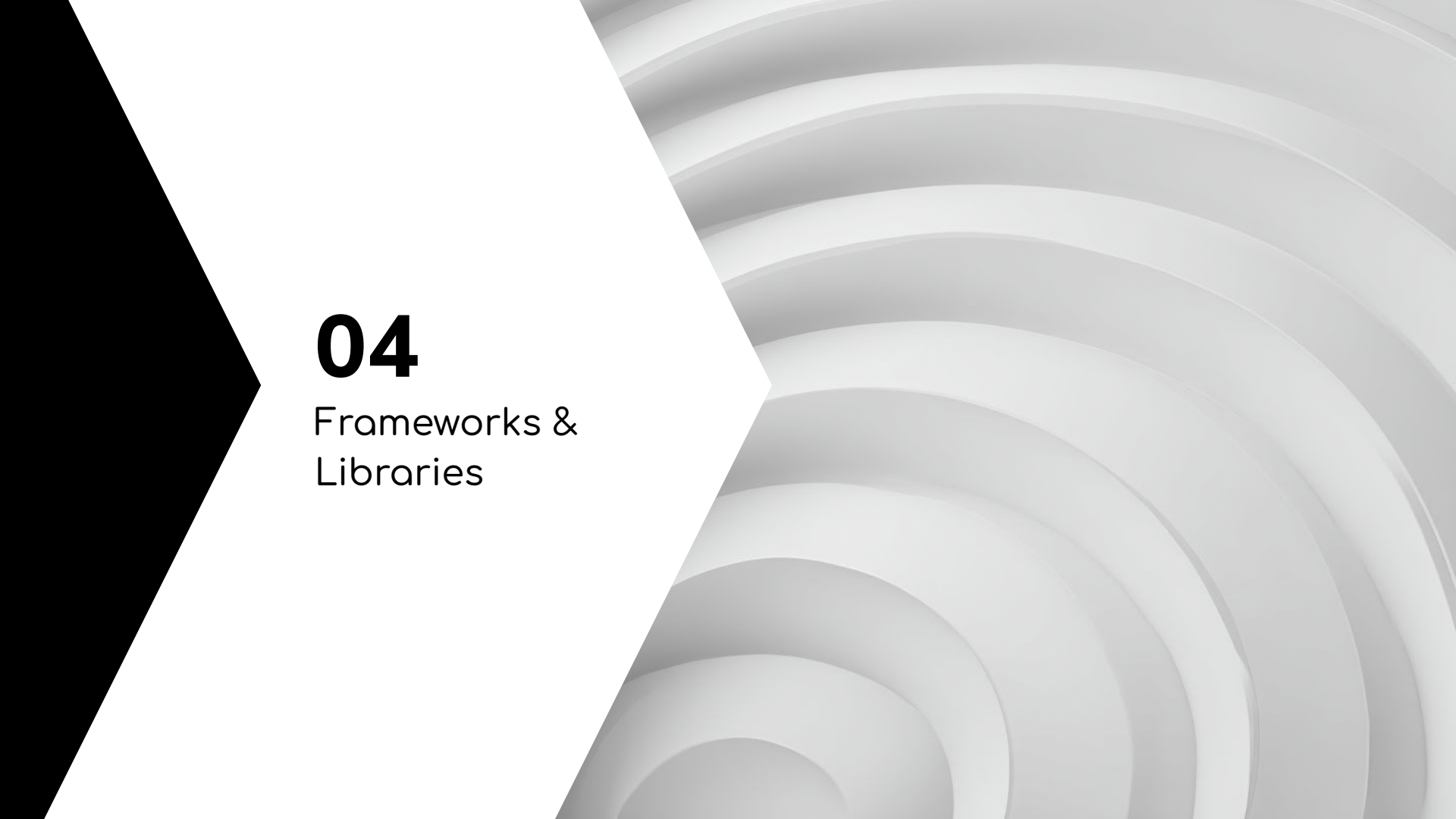


Markup Languages

Structuring Documents with Invisible Tags

A markup language is a system used to structure and format a document by adding annotations that define how content should be displayed.

These annotations are visually distinct from the content itself and are not visible when the document is processed for viewing. Common examples include HTML and XML.



04

Frameworks & Libraries



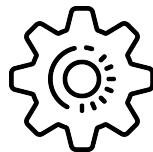
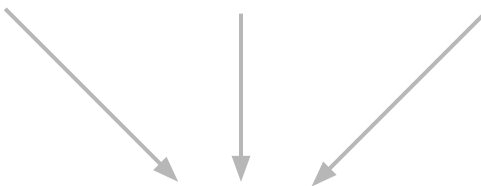
Code Library

Enhancing Projects Without Strict Rules

A code library is a collection of reusable front-end or back-end code that is "unopinionated," meaning it doesn't dictate how it should be used. Developers can integrate a library into their project and choose when and how to utilize its functionality.

Supporting Libraries

Various libraries a developer might bring into the app.



Our Application

Enhancing Projects Without Strict Rules



Library Example - Bootstrap

Simplifying Front-End Styling

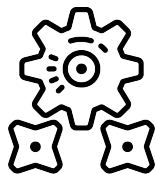
While jQuery focuses on client-side JavaScript, other libraries support front-end HTML and CSS. Bootstrap is a widely used CSS library that provides pre-styled components and responsive design utilities, along with built-in JavaScript functionality for common UI elements.



Library Example - Chart.js

Simplifying Charts for Web Applications

Chart.js is a popular JavaScript library for creating charts and graphs with ease. It simplifies the complex process of styling, layout, and data positioning, allowing developers to generate visually appealing charts quickly and efficiently.



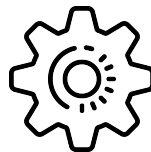
Code Framework

Structured Code for Efficient Development

A framework is a structured collection of front-end or back-end code that follows a predefined lifecycle and set of rules. Unlike libraries, frameworks are "opinionated," meaning developers build their applications to fit within the framework's structure and workflows.

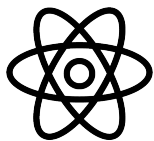
Frameworks

Has an established way of “doing things”.



Our Code

We add in our code to the framework's lifecycle.



Special Mention - React

More Than a Library, Not Quite a Framework

React is a JavaScript library developed by Facebook, designed for building dynamic user interfaces. While it has some framework-like features, it is not a full framework, as it leaves certain architectural decisions to developers. This flexibility allows for greater customization in how applications are structured.



Framework Example - Next.js

A Framework for Fast and Scalable Web Apps

Next.js is a front-end framework built on top of React (library), designed to simplify development with features like server-side rendering, static site generation, and API routes. It is an opinionated framework that streamlines performance and scalability, making architectural decisions for developers while still allowing flexibility in how applications are built.



05

Client/Server Communication



HTTP Protocol

The Backbone of Web Communication

HTTP (Hypertext Transfer Protocol) is a standard communication protocol that enables interaction between clients and servers over the internet. It ensures that different technologies used to build client-server applications can communicate seamlessly, as all modern systems include built-in support for HTTP to adhere to this universal standard.



HTTP Methods

Defining Client-Server Interactions

HTTP methods are predefined verbs that define the type of interaction a client wants to have with a server, commonly referred to as a "request." While there are 39 possible methods, the four most commonly used are GET, POST, PUT, and DELETE, which handle retrieving, sending, updating, and deleting data, respectively.



APIs

Bridging Applications Through Defined Interactions

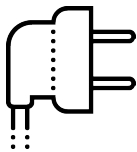
An API (Application Programming Interface) defines how an application can interact with another application, library, or framework. In the context of the web, a Web API is a set of HTTP methods that allow a client to communicate with a server or service, enabling data exchange and functionality integration.



REST APIs

Resource-Based Web Interactions

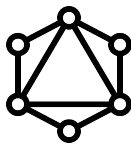
REST (Representational State Transfer) organizes an API around resources (users, orders, posts) identified by URLs. Clients use standard HTTP methods: GET to read, POST to create, PUT/PATCH to update, DELETE to remove, and servers return representations (often JSON) with status codes that explain the result.



Web Sockets

Real-Time Communication Between Client and Server

The WebSocket API enables a bi-directional, real-time communication channel between a client and server. Unlike traditional HTTP requests, WebSockets allow both the browser and server to send and receive messages instantly. This makes them essential for applications requiring live updates, such as chat platforms and real-time notifications.



GraphQL

A Smarter Way to Query APIs

Released by Facebook in 2015, GraphQL is a query language for APIs that gives developers more control over data retrieval. Unlike REST, which returns predefined data structures, GraphQL allows clients to request only the specific data they need, making API calls more efficient and flexible.



API

RPC

Beyond REST: Action-Oriented API Communication

Remote Procedure Call (RPC) is a way for one computer to ask another computer to perform a task, almost like it's running the task itself. Unlike REST, which focuses on retrieving or updating data, RPC is about telling a system to do something—like processing a payment or running a report. It helps different systems work together smoothly, especially in large, connected applications.



SOAP

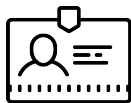
Reliable, Secure, and Still in Use

Simple Object Access Protocol (SOAP) is an older API protocol that uses XML to structure messages. It relies on Web Services Description Language (WSDL) to define what data should be exchanged and where it should be sent. While more rigid than modern alternatives like REST, SOAP is still used in systems that require strict security and reliability, such as banking and enterprise applications.



06

Authentication & Authorization



Authentication

Authentication: Verifying User Identity

Authentication helps an application confirm **who** a user is before allowing access. This can be as simple as entering a username and password or as advanced as using fingerprint scans or security codes. It's a crucial security step to protect user data and prevent unauthorized access.



Authorization

What Can You Do? Defining User Privileges

Authorization decides **what** a user can do after they log in. While authentication confirms who they are, authorization ensures they only have access to the features and information they're allowed to use. For example, a regular user might only view data, while an admin can edit or delete it.



Role-Based Access Control (RBAC)

Managing Permissions Through Roles

Role-Based Access Control is a method of controlling what users can do in a system by assigning them to predefined roles, rather than setting individual permissions for each person. For example, an “Admin” role might have full access to manage users and settings, while a “Viewer” role can only read information. RBAC is widely used in web applications, cloud platforms, and enterprise systems to ensure users only have access to the features and data they need.



Multi Factor Authentication (MFA)

Adding Extra Layers of Security

Multi-Factor Authentication is a method of protecting accounts by requiring more than just a password. Instead of relying on a single factor, MFA combines at least two different types of verification: something you know (like a password or PIN), something you have (like a phone, security token, or authenticator app), or something you are (like a fingerprint or face scan). By layering these factors together, MFA makes it significantly harder for attackers to break in, even if they manage to steal one piece of information.



OAuth and OpenID Connect

Secure Logins Made Easy

OAuth lets users log into apps and websites using accounts from services like Google or Facebook, without sharing their passwords. OpenID Connect builds on this by not only granting access but also confirming who the user is, making logins both secure and convenient.



07

Backend Frameworks



Back End Frameworks

Powering the Server Side of Web Applications

Back-end frameworks help developers build applications that work as servers, processing requests and sending responses over the internet. Some frameworks are lightweight and provide only essential features, while others include everything needed for large-scale web applications.



Microframeworks

Lightweight Frameworks with Maximum Flexibility

Microframeworks give developers a basic structure to build on while leaving many decisions open-ended. They provide just enough functionality to get started but allow flexibility in how features like databases and security are implemented.

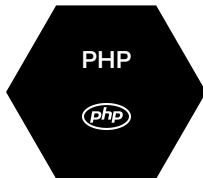
EXAMPLES OF MICROFRAMEWORKS

and the associated language

Spark
Micronaut

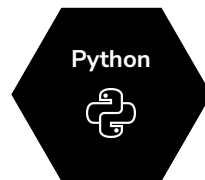


PHP

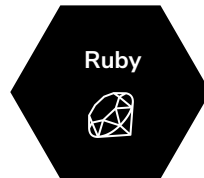


Phalcon
Lumen

Flask
FastAPI

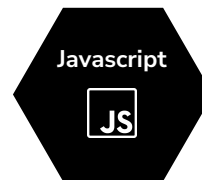


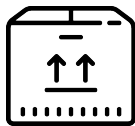
Ruby



Sinatra

ExpressJS
Koa





Modules & Packages

Pre-Built Solutions for Common Challenges

Microframeworks keep things simple, but developers still need solutions for common tasks. Modules and packages are reusable code bundles that solve these problems, making it easier to add features like databases, security, and APIs without starting from scratch.

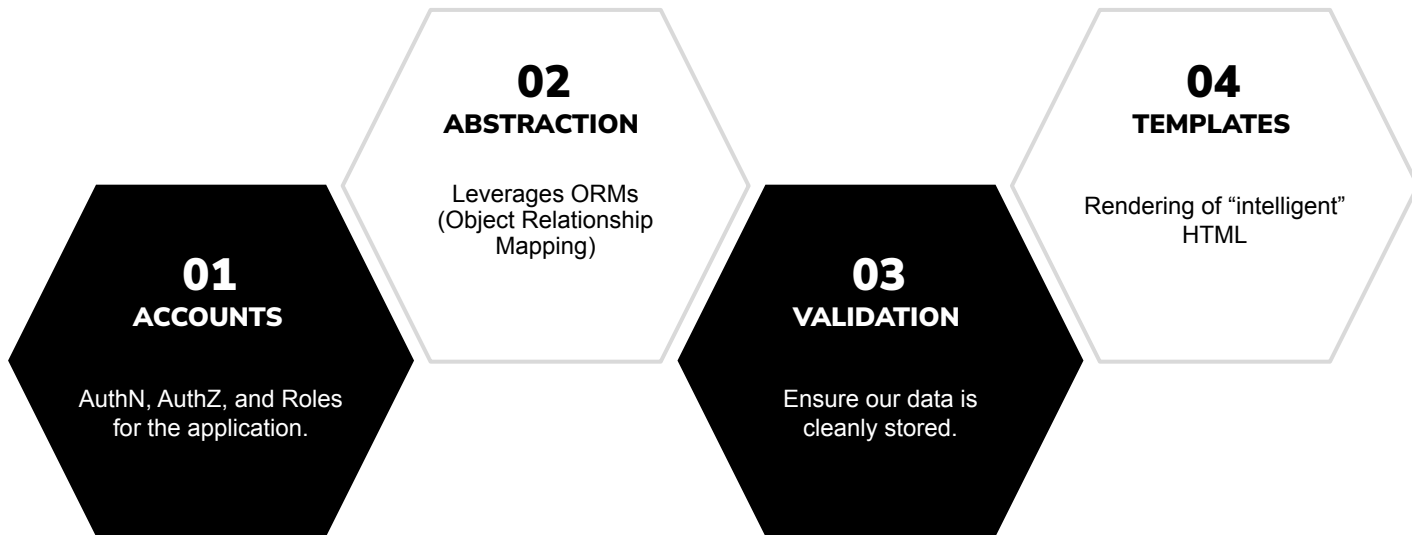


Web Frameworks

The Backbone of Scalable Web Applications

Web frameworks help developers build web applications efficiently by handling key tasks like managing data, user authentication, and security. They provide a structured approach to development, ensuring scalability and reliability. Frameworks like .NET and Spring continue to be widely used in businesses and large-scale applications.

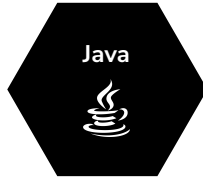
WEB FRAMEWORKS COMMON SOLUTIONS



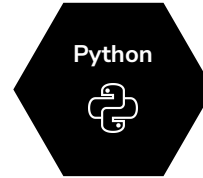
EXAMPLES OF WEB FRAMEWORKS

and the associated language

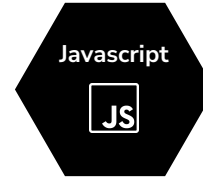
Spring
Struts



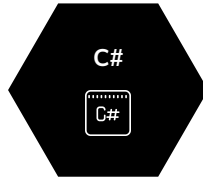
Django



Nest.js

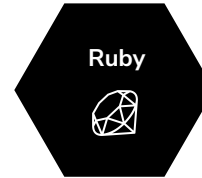


C#

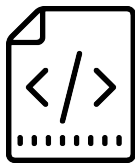


.NET

Ruby



Ruby on Rails



Template Engines

Creating Dynamic Pages with Web Frameworks

Template engines help web frameworks create HTML pages by pulling in data and logic from the server. Before web apps became more interactive, most of the processing happened on the server, and templates made it easier to organize and display information. Today, different frameworks use different template engines to generate dynamic web pages.



08

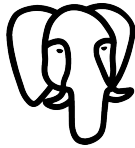
Databases



SQL - Relational Databases

Organizing Data with Structured Relationships

Relational databases store information in tables, similar to a spreadsheet, where data is organized into rows and columns. These tables can be connected, making it easy to group and retrieve related information. SQL is the language used to search, filter, and manage the data efficiently.



Relational Database Example - PostgreSQL

Trusted Database for Applications of All Sizes

PostgreSQL, or Postgres, is a free and open-source database system that organizes data in tables and follows SQL standards. It's highly flexible, supports complex queries, and can handle everything from small projects to large-scale applications.



NoSQL – Non-Relational Databases

Flexible Data Beyond Tables

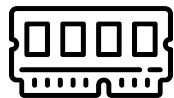
NoSQL databases break away from the traditional table-based structure of relational databases. Instead of rigid rows and columns, they offer a variety of models such as documents, key-value pairs, graphs, and wide-columns. This flexibility allows developers to store and query data in ways that better match modern application needs, particularly when dealing with large volumes of unstructured or rapidly changing information.



Document Database Example - MongoDB

Flexible, Fast, and Scalable

MongoDB is a cross-platform, document-oriented database that falls under the NoSQL category. Unlike traditional relational databases, MongoDB stores data in JSON-like documents with flexible, optional schemas. This allows for dynamic and scalable data storage, making it a popular choice for applications requiring high performance and adaptability.



In-Memory Databases

Fast Data Access, But Not Always Permanent

An in-memory database keeps data in RAM instead of saving it to a hard drive, making it much faster to access. While some in-memory databases lose data when restarted, others have features to save information permanently. They are commonly used for real-time applications, caching, and session storage to improve performance.



In-memory Database Example - Valkey

A Key-Value Store with Lightning Speed

Valkey is a super-fast in-memory database that stores data in a way that makes retrieval almost instant. It's commonly used for caching, real-time messaging, and quick lookups. While it mainly keeps data temporarily, it can be configured to save information for long-term use.



09

Developer Tools



Code Editor

Optimized for Coding, Built for Efficiency

A code editor is a tool programmers use to write and edit code. It's more advanced than a regular text editor because it includes features that make coding easier, like color-coded text, auto-suggestions, and error detection.



Visual Studio Code

The All-in-One Coding Environment

Visual Studio Code (VS Code) is a lightweight yet powerful code editor designed for modern development. It combines the simplicity of a text editor with advanced features like syntax highlighting, debugging, built-in Git tools, and an extensive library of extensions. Developers use VS Code to write, test, and manage code efficiently, all within a highly customizable workspace.



IDEs

The All-in-One Developer Toolkit

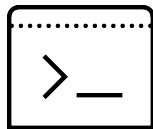
An IDE is a complete software development tool that helps programmers write and test their code. It includes a code editor, tools to build and run programs, and a debugger to fix errors, all in one place, making development more efficient.



IntelliJ IDEA (CE)

Smart Tools for Serious Coding

IntelliJ IDEA Community Edition is a free, open-source integrated development environment (IDE) built for professional-grade programming. It goes beyond a basic editor with intelligent code completion, powerful refactoring tools, and built-in support for languages like Java, Kotlin, and more. Developers use it to streamline their workflow, catch errors early, and build complex applications with confidence.



Console

Where Developers Work Behind the Scenes

The console is a text-based tool in an operating system that lets users type commands to perform tasks. While regular users rely on graphical interfaces, programmers use the console to run programs, manage files, and automate tasks more efficiently.



Package Manager

Automating Code Management

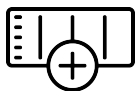
A package manager helps programmers find and install pre-built code made by other developers, so they don't have to build everything from scratch. It also keeps software up to date and organizes all the extra tools a project needs.



Build Automation

Automating the Development Workflow

Build automation tools help programmers automate repetitive tasks, like shrinking code files, converting code to different formats, and preparing applications for release. This saves time and ensures everything runs smoothly.



ORMs

Bridging Code and Databases

An ORM lets programmers work with databases using code instead of writing complex SQL queries. It turns database tables into objects, making it easier to create, read, update, and delete data using familiar programming language commands.



Postman

A User-Friendly API Testing Tool

Postman is a tool that helps developers test and interact with APIs without writing extra code. Instead of using a command line, it provides a simple interface to send requests to a server and see the responses, making API testing easier and more efficient.



10

Cloud Platforms

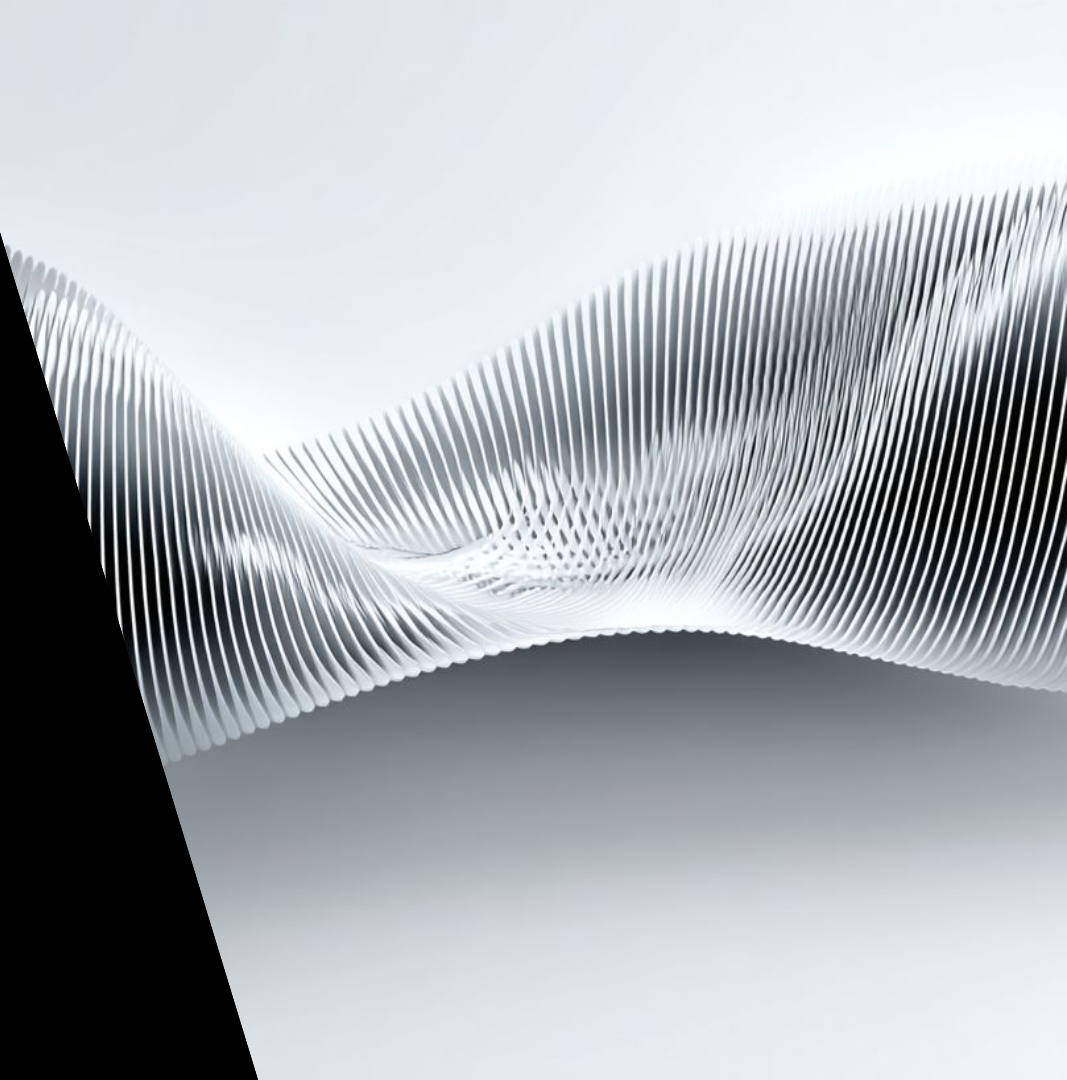
HOW APPLICATIONS RUN

Code creates Applications.

Applications run in Runtime Environments.

Runtime Environments run on Operating Systems.

Operating Systems run on Hardware.

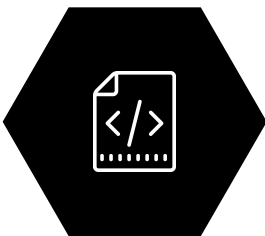


HOW APPLICATIONS RUN

An Example of a Client Application Running

1. Application runs on a...

Example: Application.js



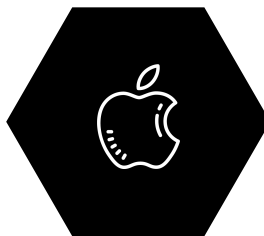
2. Runtime, That runs on an...

Example: Chrome



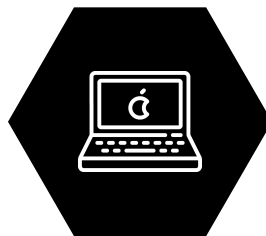
3. Operating System That runs on...

Example: macOS



4. Our Hardware

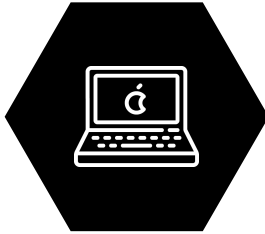
Example: MacBook



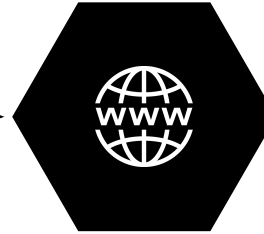
HOW APPLICATIONS RUN

An Example of how a Client Application Communicates

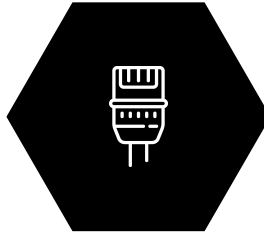
**1. Our Hardware
connects to...**



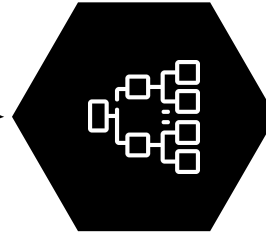
**3. Our Internet
Service Provider
connects to...**



**2. Our Wired or Wifi
Network connection
connects to...**



**4. Other Computers
wired into the internet**





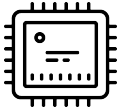
Our Computer Hardware

Processor, Memory, and Storage: The Basics

To understand the Cloud, we first need to know how our own computers work. Every computer has three key parts: the processor (CPU) that runs tasks, memory (RAM) that temporarily holds data, and a hard drive (or SSD) that stores information long-term. These parts work together to power everything we do on a computer.

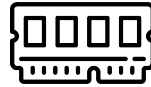
KEY HARDWARE COMPONENTS

Helpful in understanding "The Cloud"



Processor

Speed / Power
Measured in GHz and Cores



Memory / RAM

Short Term Memory
Linked to Speed
Measured in MHz and GB



Storage Space

File Storage Space
Measured in GB and TB,
Available in HDD and SSD



Cloud Platforms

More Than Just Hosting Websites

A Cloud Platform is a collection of online services that let businesses and developers run websites, store data, and manage applications without needing their own physical hardware. Over time, cloud platforms have expanded to offer a variety of powerful tools, from AI services to database management.



Cloud Platform Example: Heroku

A Beginner-Friendly Cloud Platform

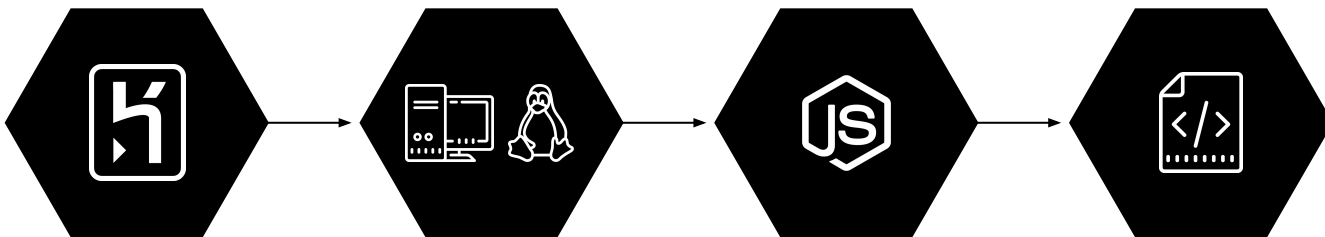
Heroku is a cloud service that makes it easy for developers to build and launch applications without worrying about managing servers. Its simple setup makes it a great choice for beginners and small teams who want to focus on coding instead of infrastructure.

CLOUD PROVIDERS TAKE CARE OF THINGS FOR US!

That way, we can focus on the code

1. Heroku provides...

3. Runtime Environment
that runs...



2. Hardware and an
Operating System that
runs a...

4. Our Application



AWS (Amazon Web Services)

A Full Suite of Cloud Services

AWS provides a huge collection of cloud services that help developers build and manage applications. Instead of using everything, teams can pick and choose the services that make the most sense for their project, from storage and computing power to databases and AI tools.



Cloud Certification

Optional, But Valuable

Cloud Certifications help professionals prove their knowledge of cloud platforms like AWS, Azure, and Google Cloud. They come in different levels and cover various topics, but not having a certification doesn't mean someone isn't skilled—some people just prefer learning through experience.



11

DevOps



DevOps

Speed, Efficiency, and Continuous Delivery

DevOps is a set of practices that bridges the gap between software development and IT operations, enabling teams to work together more efficiently. The goal is to speed up the software development lifecycle while ensuring continuous delivery and high-quality releases. By automating workflows, improving collaboration, and using modern deployment strategies, DevOps helps organizations release software faster and more reliably.



Continuous Integration

Merging Code Smoothly, Several Times a Day

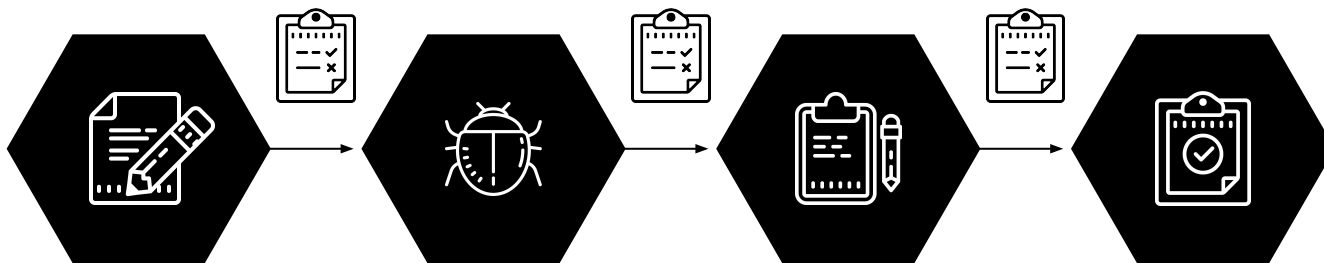
Continuous Integration (CI) is the practice of automating the building and testing of an application every time new code is committed. It ensures that all developers' code changes are regularly merged into a shared project, often multiple times a day. This process helps catch bugs early, maintain code quality, and streamline development.

TESTING ALONG THE WAY

We test code in lower environments to ensure stability by the time it is live in Production

1. Dev Environment

3. Staging Environment



2. QA Environment

4. Production



Continuous Delivery

Always Ready to Deploy

Continuous Delivery (CD) means releasing software updates more often instead of waiting long periods between changes. It helps teams quickly deliver new features and bug fixes by automating much of the process, so updates are always ready for production.



Containerization And Docker

Consistent, Portable, and Scalable Applications

Containerization is a method of packaging software along with its libraries, dependencies, and configuration files into a lightweight, portable environment (runtime) that runs consistently across different systems. Docker is a leading platform that enables this process, making it easier for developers to build, ship, and run applications without worrying about inconsistencies between environments.



Container Orchestration and Kubernetes

Scaling and Managing Containers

Container orchestration helps developers automate and manage multiple containers so they run efficiently and scale when needed. Kubernetes, or K8s, is the most popular tool for this, making it easier to deploy and maintain large applications.



Continuous Monitoring

Catching Problems Before They Grow

Continuous Monitoring is the practice of automatically tracking an application's performance, security, and reliability in real time. It helps teams detect issues early, analyze trends, and respond quickly to potential failures or security threats. Continuous monitoring is a key part of modern DevOps workflows, ensuring that applications remain stable and secure after deployment.



DevOps Maturity Model

A Roadmap for DevOps Growth

The DevOps Maturity Model helps companies measure their progress in adopting DevOps. It outlines different stages, showing where a company stands and what improvements they can make to create a more efficient and automated development process.

THANK YOU!

Let's Connect!

LinkedIn:

<https://www.linkedin.com/in/scott-bromander/>

Website:

<https://www.aftercamp.io/>

AI & Web Development Concepts for Everyone

By Scott Bromander